

# LoMix: Core Contributions, Replication Findings, and Where It Goes Next

Deliverable 2 — Technical Report

Md. Irtiza Hossain

M.S. Computer Science & Engineering, BRAC University

mohammad.irtiza.hossain@gmail.com

6 September 2026

|                 |                                                                                                                                                  |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Paper</b>    | Md Mostafijur Rahman, Radu Marculescu. <i>LoMix: Learnable Weighted Multi-Scale Logits Mixing for Medical Image Segmentation</i> . NeurIPS 2025. |
| <b>Code</b>     | <a href="https://github.com/SLDGroup/LoMix">@ 464a945</a>                                                                                        |
| <b>Data</b>     | Synapse Multi-organ (BTCV), 9 classes, $224 \times 224$                                                                                          |
| <b>Hardware</b> | GTX 960, 4 GB, capability 5.2 (Maxwell, no tensor cores)                                                                                         |
| <b>Harness</b>  | <a href="https://github.com/irtiza1999/LoMix-Reproducibility-Study">https://github.com/irtiza1999/LoMix-Reproducibility-Study</a>                |

Full setup notes, run logs, per-arm curves and reproduction commands are in the accompanying execution summary (Deliverable 1). Every number below is regenerable from the logs in the harness repository above.

## 1 What the paper does

A multi-scale decoder emits one logit map per stage. Classical deep supervision applies a loss to each and sums them with fixed, usually uniform weights. LoMix changes two things.

**Mixing.** It supervises not only the individual maps but *combinations* of them. For four decoder outputs it enumerates all 11 subsets of size  $\geq 2$  and fuses each subset under four operations — add, mul, wf (a small learned fusion module) and concat — producing 44 synthetic logit maps, each receiving its own CE + Dice loss.

**Learned weights.** Every individual map and every mixed map contributes through a learned scalar passed through `softplus`, optimised jointly with the network by the same AdamW step.

```
encoder -- decoder ---+-- p4 --+
                        |-- p3 --| subsets (11) x ops (add, mul, wf, concat)
                        |-- p2 --| |
                        +-- p1 ---+ v
                        |         44 mixed maps -- CE+Dice -- x softplus(w_op,subset)
                        |         |
                        +----- CE+Dice -- x softplus(w_i) -----+--> total loss

inference: encoder -- decoder -- p1          (mixing lives only in the loss)
```

The property that makes this attractive in practice: **all of it lives in the loss**. The mixing weights are parameters of `CombinatorialMutationsLossModule`, not of `EMCADNet`. At test time the network is unchanged — same graph, same FLOPs, same latency. Training cost rises; inference cost does not.

## 2 Why I chose this paper

It asks a question my own work keeps running into, in a different domain. **CLEAR-MoE** (under review) extracts sparse experts from a *frozen* Vision Transformer and selects which layers to

convert using a calibration-driven criterion — a decision about which internal signals to trust. **Predictive Self-Conditioning** (under review, WACV 2027) feeds a LiDAR detector’s own predictions back as conditioning and is evaluated on *calibration*, not accuracy alone, because a self-conditioning loop that trusts a wrong prediction compounds its own error. LoMix asks the same question at the supervision level: *which of my own multi-scale outputs deserve weight?* It answers with a learned scalar.

### 3 Replication design

The repository exposes `--supervision {last_layer, deep_supervision, mutation, lomix}` over one fixed network, which is a ready-made ablation ladder: the network, the data and the schedule are identical and only the loss differs.

| Arm                           | Isolates                          |
|-------------------------------|-----------------------------------|
| <code>last_layer</code>       | no deep supervision at all        |
| <code>deep_supervision</code> | per-scale losses, uniform weights |
| <code>mutation</code>         | + logit mixing, <b>unweighted</b> |
| <code>lomix</code>            | + <b>learnable</b> weights        |

`mutation`  $\rightarrow$  `lomix` is the load-bearing comparison: it isolates the learned weighting from the mixing itself. If the gain came only from having more loss terms, that pair would show it.

Constraints forced a **reduced-budget reproduction**: 20 of the repository’s default 300 epochs (the paper’s main text states no epoch budget), per-step batch 2 with gradient accumulation 3 to restore the paper’s effective batch of 6, fp32, validation every 2 epochs. Configuration was chosen by measurement, not guesswork — a memory/latency sweep showed that AMP is  $2.8\text{--}3.2\times$  *slower* on a card without tensor cores, and that the binding constraint is the Windows 2-second GPU watchdog rather than VRAM.

## 4 Findings

### 4.1 The zero-inference-overhead claim holds

| Encoder                | Params (M) | Dec. (M) | GMACs | Latency | Peak VRAM |
|------------------------|------------|----------|-------|---------|-----------|
| <code>pvt_v2_b0</code> | 3.921      | 0.507    | 0.657 | 17.7 ms | 34 MiB    |
| <code>pvt_v2_b2</code> | 26.773     | 1.914    | 4.324 | 36.8 ms | 147 MiB   |

All four arms share a byte-identical inference graph, so one measurement settles it for all of them. The decoder’s 1.914 M parameters match EMCAD’s published decoder cost, confirming the model is built as intended.

### 4.2 Training cost, which the paper does not report

Same network, same batch, only `--supervision` differs:

| Arm                           | Peak train VRAM | Step time | vs last_layer |
|-------------------------------|-----------------|-----------|---------------|
| <code>last_layer</code>       | 1008 MiB        | 649 ms    | 1.00×         |
| <code>deep_supervision</code> | 1054 MiB        | 708 ms    | 1.09×         |
| <code>mutation</code>         | 1268 MiB        | 938 ms    | 1.45×         |
| <code>lomix</code>            | 2290 MiB        | 1649 ms   | <b>2.54×</b>  |

LoMix costs  $2.54\times$  step time and  $2.27\times$  training memory, and two thirds of that gap comes from the learnable-weight arm alone. On constrained hardware this decides whether the method is runnable at all — the same class of concern the lab’s efficiency work targets, applied to training rather than inference.

### 4.3 Accuracy: the ordering did not reproduce at 20 epochs

| Arm              | Best mean Dice          | Status                            |
|------------------|-------------------------|-----------------------------------|
| last_layer       | 0.6764                  | completed                         |
| deep_supervision | <b>0.7979</b>           | completed                         |
| mutation         | 0.7788                  | completed                         |
| lomix            | 0.7225 (pre-divergence) | <b>diverged (NaN) at epoch 13</b> |

The paper’s Table 1 reports these same four arms on this same backbone (PVT-EMCAD-B2, Synapse 8-organ), averaged over at least three runs on an RTX A6000, which makes the comparison direct — though not like-for-like:

| Arm                   | Paper (Table 1) |             |             | This work         |
|-----------------------|-----------------|-------------|-------------|-------------------|
|                       | DICE            | HD95        | mIoU        | DICE              |
| LL (last_layer)       | 80.9            | 22.9        | 71.2        | 67.6              |
| DS (deep_supervision) | 82.9            | 19.7        | 73.8        | <b>79.8</b>       |
| mutation              | 83.6            | 15.7        | 74.7        | 77.9              |
| lomix                 | <b>85.1</b>     | <b>14.9</b> | <b>76.4</b> | 72.3 <sup>†</sup> |

Percentages. <sup>†</sup>best before divergence at epoch 13, not a completed run.

The published ordering is monotone; mine is not. My absolute values are uniformly lower, and my best arm (79.8) sits just below the paper’s *weakest* arm (80.9) — what 20 epochs against a converged schedule looks like, rather than a broken setup. The margins at stake are also small: the paper’s DS  $\rightarrow$  LoMix gap is 2.2 DICE, established over at least three runs with a two-sided Wilcoxon signed-rank test. One seed at 20 epochs cannot resolve an effect that size.

Two further observations. `last_layer` peaks at its *first* validation and decays thereafter, while the supervised arms climb steadily — consistent with §4.5. And the learned mixing weights had barely differentiated: from a common initialisation of  $\text{softplus}(0) = 0.693$ , the four per-scale weights ended at 0.379, 0.386, 0.392, 0.400. The mechanism supposed to create LoMix’s advantage had not yet engaged at this budget, which is the most likely explanation for the ordering and is not separable from it with one seed at 20 epochs.

### 4.4 The lomix arm diverged, and the cause is in the loss

At iteration 14900, epoch 13, all loss terms became NaN simultaneously and every later validation returned 0.000. Not a hardware event: fp32 throughout, no OOM, no watchdog reset, and the saved weights are finite. Two defects in the combinatorial loss explain it, both verified empirically rather than read off the source (`tools/diagnose_loss_ops.py`).

`mul` **multiplies raw logits**, so a  $k$ -map combination is a  $k$ -fold product growing like  $|\text{logit}|^k$ . Measured on the trained checkpoint’s own logits ( $\max |p| = 6.16$ ):

| Operation | $k = 2$     | $k = 3$      | $k = 4$       |
|-----------|-------------|--------------|---------------|
| add       | 12.0        | 17.9         | 23.6          |
| mul       | <b>36.2</b> | <b>211.0</b> | <b>1214.0</b> |
| wf        | 6.1         | 6.1          | 5.9           |
| concat    | 2.8         | 2.9          | 1.9           |

CE and Dice saturate at that magnitude and the gradient overflows. Nothing clips or normalises the product. This is a property of the method as released, not of this hardware.

**concat rebuilds a random convolution on every forward pass.** The  $1 \times 1$  conv that fuses the concatenated maps is constructed *inside forward*, randomly initialised each call, never registered as a submodule and never given to the optimizer. Calling the loss twice with identical inputs under `no_grad`:

| Operation      | Max difference between two identical calls |
|----------------|--------------------------------------------|
| add / mul / wf | 0.0 (deterministic)                        |
| concat         | <b>5.41 (non-deterministic)</b>            |

None of the loss module’s 66 parameter tensors belongs to it. So the `concat` branch cannot learn and injects a fresh random projection into the decoder each step. This predicts exactly what the weight readout shows: `concat`’s eleven learned weights collapsed to a common 0.3755–0.3770 and stopped moving, because there is no gradient signal for them to differentiate on. Of the four operations, one is inert by construction and another is numerically unbounded.

#### 4.5 Three further defects in the released code

- **The training script cannot start.** `train_synapse_lomix.py:10` imports `PVT_CASCADE`, defined nowhere in the repository; the only other reference is a commented-out line.
- **Training continues in eval mode.** `model.train()` is called once before the epoch loop, `inference()` calls `model.eval()` after every epoch, and nothing switches back — so from epoch 1 the decoder’s BatchNorm uses frozen running statistics. The same pattern appears in EMCAD and G-CASCADE. Left intact here, since the published results were produced by this code; exposed behind an opt-in flag for a separate experiment.
- **Windows portability.** `worker_init_fn` is a local function, which `spawn` cannot pickle, so multi-worker loading fails on Windows and works on Linux.

### 5 Limitations of this replication

Single seed, 20 of the repository’s default 300 epochs, one dataset, one encoder, no statistical testing. The `deep_supervision/mutation` gap (0.798 vs 0.779) is well inside what one seed can produce. What these runs **do** support independently of budget: the inference-cost claim (measured directly), the training-cost ratios (measured under identical conditions), and the two loss defects (verified on the model’s own logits, not dependent on training length). What they **do not** settle is whether LoMix outperforms uniform deep supervision at full budget — that needs the full-schedule run this hardware cannot deliver.

### 6 Where I would take it

Two directions follow directly from the findings, and the first is my Step 2 proposal.

**Uncertainty-conditioned mixing weights.** LoMix learns one global answer to “which scale do I trust”, then freezes it. §4.3 shows those weights differentiate very slowly; §4.2 shows enumerating all 44 combinations is what makes training expensive. Conditioning the weights on a per-sample uncertainty estimate addresses both: it gives the weights a richer signal than a single global scalar, and it offers a principled basis for *selecting* which subsets are worth supervising rather than enumerating all of them — cutting cost on precisely the axis that dominates. Because the weights live in the loss, the inference graph stays untouched, preserving the property that makes LoMix deployable. Evaluated on calibration (ECE, reliability) as

well as Dice, this connects directly to the calibration-driven selection in CLEAR-MoE and the confidence-aware objective in my LiDAR detection work.

**Bounded mixing operations.** Multiplying softmax probabilities rather than raw logits, or normalising the product, would remove the divergence mode in §4.4 at no representational cost — and registering the `concat` conv properly would make a quarter of the operation set functional for the first time. Both are small changes with a measurable prediction attached: if `concat` is currently inert, fixing it should change its learned weights away from the collapsed value.

## Appendix — slide outline (6 slides)

1. **LoMix in one slide** — decoder logits  $\rightarrow$  subset mixing  $\rightarrow$  learned weights; loss-only, therefore zero inference overhead.
2. **Why this paper** — CLEAR-MoE (calibration-driven selection) and PSC (self-conditioning judged on calibration) ask the same question.
3. **Setup** — four arms, one network; the 4 GB deviation table; AMP measured slower, TDR watchdog as the real constraint.
4. **What reproduced** — efficiency table; training-cost table ( $2.54\times$ ).
5. **What did not** — ablation table, divergence at epoch 13, and the two loss defects with the `mul` magnitude and `concat` determinism evidence.
6. **Next** — uncertainty-conditioned weights, bounded operations  $\rightarrow$  Step 2 proposal.